
be satisfied simultaneously. This finding directly informs our test design, where several prompts deliberately combine multiple constraints to assess comprehensive adherence.

Skopek et al. [14] provide a detailed case study in summarization tasks, demonstrating that even in well-studied domains, instruction-following failures persist when specific constraints are introduced. Their work highlights the distinction between task capability and instruction adherence—a model may excel at summarization while failing to respect specified length or content constraints.

Liu et al. [9] recently conducted a re-evaluation of instruction-following benchmarks, identifying systematic issues in existing evaluation methodologies and proposing refinements. Their work underscores the evolving nature of instruction-following evaluation and the need for continued methodological development.

Efficiency-Focused Evaluation Approaches

The need for efficient evaluation methods has been recognized in various contexts. Benchmark compression techniques, such as those proposed by Polo et al. [11], attempt to identify minimal subsets of tasks that preserve ranking information. Their tinyBenchmarks approach demonstrates that carefully selected subsets can maintain evaluation validity while reducing computational requirements. Our approach differs by focusing on diagnostic capability rather than ranking preservation, prioritizing the identification of specific failure modes over comprehensive performance scoring.

Wang et al. [16] conducted an extensive analysis of instruction tuning on open resources, evaluating numerous models across various benchmarks. Their work demonstrates the value of comprehensive empirical analysis while highlighting the practical challenges of large-scale evaluation, further motivating the development of efficient diagnostic frameworks like ours.

Materials and Methods

Our evaluation framework employs a systematic approach to assess instruction-following capabilities across a diverse range of large language models. This section details the design principles, test prompt construction, evaluation infrastructure, and measurement criteria used in our study.

0.1 Test Prompt Design

We developed twenty diagnostic prompts specifically engineered to evaluate distinct aspects of instruction adherence. Each prompt incorporates verifiable constraints that enable objective, automated assessment of compliance. Our prompt selection criteria prioritized three key characteristics: unambiguous success criteria, minimal computational requirements for verification, and coverage of diverse instruction types commonly encountered in practical applications.

The prompt categories include:

Sequential Instruction Execution: These prompts require models to perform multiple steps in a specified order, testing their ability to maintain task coherence across sequential operations. For example, our first test requires extracting letters from a word, reversing them, and applying specific formatting—each step building upon the previous result.

Constrained Text Generation: This category evaluates models’ ability to generate content while adhering to explicit constraints. Our second test exemplifies this by requiring factual content about a well-known subject while completely avoiding a

common letter, testing both content generation and constraint maintenance simultaneously.

Structured Output Formation: These prompts assess whether models can produce outputs in precise formats, particularly important for downstream system integration. Our third test requires analyzing a word list, applying filtering criteria, and outputting results in valid JSON format with specific formatting requirements.

0.2 Evaluation Infrastructure

We implemented our evaluation framework using the OpenRouter API, which provides unified access to multiple language model providers through a single interface. This approach offers several advantages: consistent API structure across different model families, simplified authentication and billing, and access to models that might otherwise require separate implementations for each provider.

The evaluation system executes each test systematically across all target models, maintaining consistent parameters to ensure comparability. We set temperature to 0.0 to minimize response variability and enable more deterministic evaluation of instruction-following capabilities. Response timeout limits were established at 10 seconds to balance comprehensive model coverage with practical execution constraints.

0.3 Data Collection and Analysis

Our framework captures multiple dimensions of model performance for each test execution. Primary metrics include binary pass/fail determination based on adherence to specified instructions, response time measurement for performance characterization, and token usage statistics for efficiency analysis.

Results are automatically compiled into a structured Excel workbook with multiple analytical views. The overview sheet provides a high-level pass/fail matrix across all models and tests, enabling rapid identification of performance patterns. Individual test sheets contain detailed response data, allowing deeper investigation of specific failure modes. A model summary sheet aggregates performance statistics, including success rates, average response times, and total token consumption.

0.4 Verification Methodology

Each test prompt includes programmatically verifiable success criteria. Our verification approach accounts for common variations in model outputs while maintaining objective assessment standards. For instance, when evaluating JSON output formatting, the system handles variations such as presence or absence of whitespace, different quote styles, and markdown code block wrappers. This flexible verification reduces false negatives while preserving the integrity of the evaluation.

The verification process operates in two stages: primary verification applies strict matching criteria to determine exact compliance, while secondary verification considers acceptable variations that maintain semantic correctness. This dual-stage approach provides both rigorous assessment and practical acknowledgment of stylistic variations across different model implementations.

0.5 Model Selection and Verification

To ensure unbiased model selection and prevent confounding effects from non-functional API endpoints, we employed a rigorous two-stage verification and selection protocol.

Stage 1: Comprehensive Endpoint Verification. On October 14, 2025, we queried the OpenRouter API to retrieve all available models, yielding a total of 331

models across diverse providers and architectures. To prevent selection bias arising from non-functional endpoints or implementation issues that could compromise our instruction-following evaluation, we first verified each model’s basic responsiveness with a simple factual query: "What is the capital of France?"

This verification step serves a critical methodological purpose: if a model cannot provide a simple factual response, its failure on complex instruction-following tasks would be confounded by fundamental endpoint issues rather than reflecting genuine instruction adherence deficits.

We employed a parameter-aware testing protocol that respects each model’s API specification. Prior to testing, we retrieved comprehensive metadata for each model via the OpenRouter API, including its `supported_parameters` field which specifies which generation parameters (e.g., `temperature`, `max_tokens`, `seed`) each model accepts. Our verification payload was constructed dynamically for each model to include only its supported parameters. Models received `temperature=0.0` and `max_tokens=150` where supported, and `seed=42` for reproducibility where supported. This parameter-aware approach prevents API errors from unsupported parameters while ensuring each model is tested under appropriate conditions.

Note on the `reasoning` parameter: While many models support a `reasoning` parameter according to API metadata, we excluded this parameter from verification testing after determining it requires a specific object format rather than a simple string value. This decision proved critical in preventing false negatives due to API parameter format issues rather than genuine model unavailability.

Models were classified as verified working if they: (1) returned a valid API response within timeout (30 seconds), (2) provided coherent text content that included "Paris" in the response. The verification process identified 256 working models (77.3% of 331 total) that successfully responded with the correct answer. Models that failed verification included those with endpoint errors, timeout errors, rate limiting on free tiers, or incorrect responses. This rigorous verification ensures that our instruction-following evaluation focuses on models with confirmed functional API access.

Stage 2: Model Selection for Evaluation. From the 256 verified working models identified in Stage 1, we evaluated all models to ensure comprehensive coverage across the contemporary LLM landscape. This inclusive approach, made feasible by our efficient 20-prompt framework, eliminates sampling bias and provides maximum statistical power for detecting instruction-following patterns. The evaluation set comprises models from major commercial providers (OpenAI, Anthropic, Google, Meta, Mistral) as well as emerging and open-source implementations (Qwen, DeepSeek, and community models), spanning parameter counts from sub-billion to hundreds of billions. This extensive coverage, combined with our verification protocol, enables meaningful comparison across the current landscape of language model capabilities while ensuring that observed instruction-following patterns reflect genuine model characteristics rather than implementation or availability issues.